C. elegans SWIP Tracking Suite Master Documentation: Explanation and Validation of PySWIP, PySWIPR, and BATRR

What this document is. This guide explains, step by step, how three computer programs work together to automatically count how fast a tiny worm is thrashing in a video. It is written for readers who are familiar with biology but may not have a background in computer vision or signal processing. Every technical concept is explained before it is used, and mathematical equations are accompanied by plain-language interpretations.

Contents

1 Background: What Are We Trying to Do? 3
- 1.1 The Worm and the Assay . . . 3
- 1.2 The Problem with Doing This by Hand . . . 3
- 1.3 The Three-Program Pipeline . . . 3

2 PySWIP: Watching the Video and Measuring the Worm 3
- 2.1 Step 1: Fixing the Lighting (Illumination Correction) . . . 4
  - 2.1.1 The Problem . . . 4
  - 2.1.2 Stage 1: Background Division — Flattening the Dome . . . 4
  - 2.1.3 Stage 2: CLAHE — Boosting Local Contrast . . . 4
- 2.2 Step 2: Building and Maintaining a Background Model . . . 5
  - 2.2.1 The Core Idea . . . 5
  - 2.2.2 The Exponential Moving Average . . . 5
  - 2.2.3 The Problem: Stationary Objects Get Absorbed . . . 5
  - 2.2.4 The Solution: Freeze Masks . . . 6
  - 2.2.5 Paralysis-Safe Initialisation . . . 6
- 2.3 Step 3: Finding the Worm (Dual-Threshold Segmentation) . . . 6
  - 2.3.1 The Difference Image . . . 6
  - 2.3.2 Why One Threshold Is Not Enough . . . 7
  - 2.3.3 The Dual-Threshold Flood-Fill Strategy . . . 7
- 2.4 Step 4: Tracing the Worm's Body (Spine Extraction) . . . 7
  - 2.4.1 From Blob to Contour . . . 7
  - 2.4.2 Resampling the Contour . . . 7
  - 2.4.3 Finding the Head and Tail . . . 7
  - 2.4.4 Building the Midline Spine . . . 8
- 2.5 Step 5: Measuring the Body Shape . . . 8
  - 2.5.1 The Bending Angle θ . . . 8
  - 2.5.2 Other Shape Measurements . . . 8
- 2.6 Step 6: Keeping Track of Objects Across Frames . . . 9
  - 2.6.1 Why Tracking Is Necessary . . . 9
  - 2.6.2 Matching Blobs to Existing Tracks by Overlap . . . 9
  - 2.6.3 Preventing False New Detections . . . 9

2.6.4 Keeping Head and Tail Consistent . . . 10
2.7 Output: The Raw Data Spreadsheet . . . 10

3 PySWIPR: Cleaning the Data **11**
3.1 Stage 1: Remove Physically Impossible Values . . . 11
3.2 Stage 2: Remove Short-Lived Objects . . . 11
3.3 Stage 3: Resolve Overlapping Tracks . . . 12
3.4 Stage 4: Remove Data-Sparse Time Windows . . . 12

4 BATRR: Counting Thrashes from the Angle Signal **13**
4.1 Why Not Just Count Zero-Crossings of the Angle? . . . 13
4.2 Computing Angular Velocity . . . 13
4.3 Counting Peaks in the Velocity Signal . . . 13
4.4 Converting Count to Rate . . . 14

5 Implementation Details **15**
5.1 Software Requirements . . . 15
5.2 Data Files Produced at Each Stage . . . 15

6 Pipeline Summary and TikZ Diagram **16**

7 Pipeline Validation: Comparison with Manual Counting **18**
7.1 Shapiro-Wilk Test for Normality . . . 18
7.2 Linear Regression Analysis . . . 18
7.3 Bland-Altman Analysis . . . 19
7.4 Passing-Bablok Regression Analysis . . . 20
7.5 Algorithm Performance Assessment Summary . . . 20

8 References **21**

9 Supplementary Material **22**
9.1 Standard Deviation of Angles: Worms and Debris . . . 22
9.1.1 Shapiro-Wilk's Test for Normality . . . 22
9.1.2 Outlier Identification and Retention . . . 22
9.1.3 Levene's Test and Welch's ANOVA . . . 22
9.1.4 Games-Howell Post Hoc Analysis . . . 23
9.1.5 Statistical Analyses Overview . . . 23
9.1.6 Conclusion . . . 23

1 Background: What Are We Trying to Do?

1.1 The Worm and the Assay

C. elegans is a microscopic roundworm about 1 mm long. It is one of the most studied animals in biology because its entire nervous system — exactly 302 neurons — has been mapped, and its genes are easy to manipulate. When a healthy *C. elegans* is placed in liquid, it thrashes rhythmically: it bends its body from side to side in a smooth, repeating wave, much like a swimmer doing the butterfly stroke. A healthy worm does this roughly 30–50 times per minute.

The Swimming-Induced Paralysis (SWIP) assay exploits this behaviour. When certain genes are broken — particularly those involved in dopamine signalling — worms cannot thrash properly and become paralysed in liquid. Similarly, drugs that affect these pathways change the thrash rate. By measuring how many times per minute a worm thrashes, researchers get a precise, quantitative number that reflects how well the worm's nervous system is functioning.

1.2 The Problem with Doing This by Hand

A researcher watching a video can count thrashes manually. But a modern genetic screen might involve thousands of worm strains across hundreds of videos, each containing dozens of individual wells. Manual counting at that scale is:

- **Slow** — a single video can take hours to score.
- **Inconsistent** — different observers count differently, and the same observer counts differently on different days.
- **Time consuming to scale** — for studies aiming to increase output or track worms over long periods of time, manual counting is labor-intensive and time-consuming.

The SWIP Tracking Suite solves this by automating the entire process from raw video to thrash rates per minute.

1.3 The Three-Program Pipeline

The suite is split into three programs that run in sequence. Think of them as an assembly line:

1. **PySWIP** (written in Python) watches each video frame by frame, finds the worm in each well, measures the shape of its body, and writes all the measurements to a spreadsheet.
2. **PySWIPR** (written in R) reads that spreadsheet and cleans it up, removing measurements that are clearly wrong or come from debris rather than worms.
3. **BATRR** (written in R) reads the cleaned spreadsheet, looks at how the body angle changes over time, and counts how many thrashes occurred per minute.

The input is a single video file (`.avi` or `.mp4`) of a multi-well plate. The final output is a small spreadsheet called `thrash.csv` with one row per well per time window, containing the thrash rate in thrashes per minute (TPM).

2 PySWIP: Watching the Video and Measuring the Worm

What PySWIP does in one sentence. For every frame of the video, for every well the user has selected, PySWIP finds any worm-shaped objects, traces a line down the middle

of each one, and records 30 numbers describing its shape and position.

PySWIP works through five distinct steps, applied to every frame of the video. Each step is explained below, starting from the raw pixel data and ending with a row of numbers in a spreadsheet.

2.1 Step 1: Fixing the Lighting (Illumination Correction)

2.1.1 The Problem

A camera pointed at a multi-well plate does not illuminate all wells equally. The centre of the image tends to be brighter than the edges, and even within a single circular well the light forms a dome shape — brighter in the middle, dimmer toward the rim. This is a problem because the worm detection step later works by looking for pixels that are brighter or darker than the background. If the background itself has a gradient, the detector will generate false alarms at bright edges and miss real worms in dark corners.

2.1.2 Stage 1: Background Division — Flattening the Dome

The fix is to estimate the dome shape and divide it out. This is done by blurring the image with an extremely large blur kernel. The key insight is **scale separation**: a worm body is roughly 10 pixels wide, while the illumination dome changes slowly over the entire well diameter of ∼160 pixels. A blur kernel larger than the worm but smaller than the well will average the worm body out of existence while preserving the dome shape.

Analogy. Imagine a photograph of a table with a candle at one end, making one side brighter. To correct for this you could estimate the lighting by squinting until the table details blur away, leaving only the brightness gradient — then divide the photo by that gradient. Details (plates, cutlery) would survive; only the uneven lighting would be removed.

The kernel size is chosen automatically based on the well size:

$$k = \max\left(51,\ \left\lfloor \frac{\min(H,W)}{4} \right\rfloor \,\middle|\, 1\right), \tag{1}$$

where H and W are the height and width of the well region in pixels. The bitwise-OR with 1 simply ensures k is odd, which is a technical requirement of the blur function. The resulting blurred image $L(x, y)$ is the illumination estimate. The corrected image $C(x, y)$ is:

$$C(x, y) = \text{clip}\left(\frac{I(x, y)}{\max(L(x, y),\ 1)} \cdot \bar{L},\ 0, 255\right), \tag{2}$$

where $I(x, y)$ is the raw pixel value, $L(x, y)$ is the blurred illumination estimate, and $\bar{L}$ is the average of L across the whole well. Multiplying by $\bar{L}$ after the division puts the brightness back into a reasonable range (otherwise the result would be near 1.0 everywhere instead of near 128). The $\max(\cdot, 1)$ in the denominator prevents division by zero in very dark pixels.

In plain terms: every pixel is divided by "how bright it should be if there were nothing there," making the whole well uniformly bright.

2.1.3 Stage 2: CLAHE — Boosting Local Contrast

After background division, the large-scale dome is gone but fine-scale contrast may still be uneven. CLAHE (Contrast Limited Adaptive Histogram Equalisation) addresses this. It divides the image into small tiles and independently stretches the contrast within each tile, so that dim

regions become more visible without overexposing bright regions. The "clip limit" of 2.0 prevents the amplification of noise in tiles that are nearly uniform.

The output of these two stages — a spatially uniform, locally high-contrast image — is what all subsequent steps work with.

```
def correct_illumination(roi_gray):
    h, w  = roi_gray.shape
    # Stage 1: estimate illumination dome with large blur
    k     = max(51, (min(h, w) // 4) | 1)   # adaptive odd kernel
    bg    = np.maximum(
                cv2.GaussianBlur(roi_gray, (k, k), 0).astype(np.float32),
                1.0)                          # L(x,y), clamped to >= 1
    corr  = np.clip(
                (roi_gray.astype(np.float32) / bg) * float(np.mean(bg)),
                0, 255).astype(np.uint8)      # C(x,y)
    # Stage 2: boost local contrast with CLAHE
    tile  = max(4, min(h, w) // 16)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(tile, tile))
    return clahe.apply(corr)
```

Listing 1: The illumination correction function. The two stages (background division, then CLAHE) are applied sequentially to every well ROI.

2.2 Step 2: Building and Maintaining a Background Model

2.2.1 The Core Idea

Once the lighting is corrected, the next question is: which pixels contain the worm? The strategy is **background subtraction**: build a reference image of what an empty well looks like, then subtract it from the current frame. Pixels that are very different from the background are likely to be the worm.

The reference image — the background B_t — is not a single static snapshot. It is a running average that updates slowly over time, allowing it to adapt to gradual changes like condensation on the plate or slow camera drift.

2.2.2 The Exponential Moving Average

The background is updated every frame using an exponential moving average (EMA), a weighted blend of the old background and the current frame:

$$B_{t+1}(x, y) = (1 - \alpha)\, B_t(x, y) + \alpha\, I_t(x, y), \quad \alpha = 2^{-5} = 0.03125. \tag{3}$$

Analogy. Think of the background as your memory of what the well normally looks like. Each new frame you see, you update your memory slightly: 96.875% old memory, 3.125% new observation. Something that appears for just one frame barely changes your memory. Something that sits there for hundreds of frames gradually becomes "the new normal."

The value $\alpha = 1/32$ was chosen so that the background adapts over roughly one second (about 32 frames at 30 fps) — fast enough to track slow drift, slow enough to ignore rapid motion.

2.2.3 The Problem: Stationary Objects Get Absorbed

There is a serious flaw in naive background EMA: if a worm stops moving, the background slowly "learns" the worm's pixel values and the worm becomes invisible. After about 32 frames

($1/\alpha$ frames), the worm would be fully absorbed into B_t and the subtraction would produce zero difference everywhere.

This is especially critical for paralysed worms, which are the very thing the assay is designed to detect.

2.2.4 The Solution: Freeze Masks

PySWIP uses two complementary mechanisms to prevent this absorption.

Active-track freeze. Once a worm has been detected and is being tracked, its pixel locations are recorded as a binary mask. Those pixels are excluded from the EMA update:

$$B_{t+1}(x,y) = \begin{cases} B_t(x,y) & \text{if } (x,y) \in \mathcal{F}_t \quad \text{(frozen)}, \\ (1-\alpha)B_t(x,y) + \alpha I_t(x,y) & \text{otherwise.} \end{cases} \tag{4}$$

$\mathcal{F}_t$ is the set of all pixels currently covered by a tracked object. Those pixels are "frozen" — the background there never changes, so the worm always remains visible against it.

Ghost-mask freeze. What happens when a worm briefly disappears from detection — for example, because it coiled up and became too round to be segmented? If the freeze immediately lifts, the background starts learning the worm's pixel values during that gap. By the time the worm is detected again, the signal has collapsed and it looks like a brand new object, receiving a new tracking ID.

To prevent this, when a track dies the last known pixel mask is kept frozen for a further 60 frames (the "ghost" period). This covers the gap and prevents the background from absorbing the worm during brief detection failures.

2.2.5 Paralysis-Safe Initialisation

There is one more edge case: what if the worm is already paralysed in frame 0? In that case, the very first frame used to initialise B_0 already contains the worm. The background is initialised to the worm-containing image, the subtraction immediately produces zero everywhere the worm sits, and the worm is never detected at all.

The fix is to initialise B_0 not from the raw reference frame, but from a heavily blurred version of it, using the same large kernel from Equation (1). Because a worm body ($\sim$10 px wide) is much narrower than the kernel ($\sim$51 px), the worm is blurred out of the background estimate entirely, while the broad illumination field is preserved. The worm then appears as a clear signal against this blurred background from frame 1 onwards, regardless of whether it is moving.

2.3 Step 3: Finding the Worm (Dual-Threshold Segmentation)

2.3.1 The Difference Image

With the background estimate in hand, the foreground signal is:

$$D(x,y) = \text{GaussBlur}_{5\times5}\big(\big|I_t(x,y) - B_t(x,y)\big|\big)\,. \tag{5}$$

This is the absolute difference between the current corrected frame and the background, smoothed by a small 5×5 Gaussian blur to suppress single-pixel camera noise. Bright pixels in D indicate regions that differ strongly from the background — i.e., where the worm is.

2.3.2 Why One Threshold Is Not Enough

The simplest approach would be to apply a single threshold T: any pixel with $D(x, y) \geq T$ is declared foreground. The problem is that the worm's body does not have uniform brightness in the difference image. The dense core of the body produces a strong signal ($D \gg T$), while the thin outer edges produce a weaker signal that may fall just below T. A single strict threshold clips the edges, making the detected blob smaller than the true worm and deforming the contour used to fit the spine.

2.3.3 The Dual-Threshold Flood-Fill Strategy

PySWIP uses a two-threshold approach adapted from the original Multi-Worm Tracker (MWT). Two maps are computed:

$$\mathcal{S} = \{(x, y) : D(x, y) \geq T\} \qquad \text{(seed map: "definitely foreground")}, \tag{6}$$

$$\mathcal{R} = \{(x, y) : D(x, y) \geq \beta_T T\} \qquad \text{(relaxed map: "possibly foreground")}, \tag{7}$$

where $\beta_T = 0.8$, so the relaxed threshold is 80% of the strict threshold. Connected components (contiguous regions of white pixels) are found separately in each map. Then, for each seed-map component, all relaxed-map components that touch it are merged into it. The result is a blob that has a strict core but expanded, more complete edges.

> **Analogy.** Imagine marking a photograph with two colours: red for "certainly the object" (bright pixels) and orange for "probably the object" (slightly dimmer pixels). You start with the red regions as seeds, then grow each seed outward into any touching orange region. The final blobs have red cores with orange borders, capturing the full extent of each object.

After merging, blobs that are too small (noise) or too large (two worms stuck together or a smear) are discarded using the user-configured size bounds $[S_{\min}, S_{\max}]$. When a worm is already being tracked, these bounds are slightly relaxed (multiplied by $\beta_S = 0.7$ at the lower end and divided by β_S at the upper end) to accommodate momentary deformations during a coiling manoeuvre.

2.4 Step 4: Tracing the Worm's Body (Spine Extraction)

2.4.1 From Blob to Contour

Each accepted blob is a binary image (white pixels = worm, black = not worm). To get the worm's outline, PySWIP runs OpenCV's `findContours` function, which traces the boundary of the white region and returns it as an ordered list of (x, y) coordinate pairs. If multiple contour fragments are found (from noise or partial occlusion), only the largest one is kept.

2.4.2 Resampling the Contour

The raw contour from OpenCV has variable point density — more points where the boundary is irregular, fewer on smooth stretches. To make downstream geometry calculations well-behaved, the contour is resampled to exactly $N_c = 120$ uniformly spaced points along the arc length. A circular Gaussian smooth ($\sigma = 2$ points) is then applied to remove sub-pixel jitter, treating the contour as a closed loop.

2.4.3 Finding the Head and Tail

With a smooth, evenly sampled contour, the two tips of the worm (head and tail) are found by the furthest-pair heuristic:

1. Find the point on the contour furthest from the contour's centroid — this is one tip (call it a).
2. Find the point on the contour furthest from a — this is the other tip (call it b).

Mathematically:

$$a = \arg\max_i \|\mathbf{p}_i - \bar{\mathbf{p}}\|_2, \tag{8}$$

$$b = \arg\max_i \|\mathbf{p}_i - \mathbf{p}_a\|_2, \tag{9}$$

where $\bar{\mathbf{p}}$ is the mean position of all contour points. The symbols $\|\cdot\|_2$ simply mean Euclidean distance — the straight line between two points. The assignment of a and b to "head" and "tail" is kept consistent across frames by comparing with the previous frame's tip positions (see Section 2.6).

2.4.4 Building the Midline Spine

Once the two tips are located, the contour is split into two arcs: one path along the top of the worm from head to tail, and one path along the bottom. Each arc is independently resampled to $N_s = 11$ equally spaced points. The midline spine is the pointwise average of the two arcs:

$$\mathbf{s}_k = \frac{\mathbf{a}_k + \mathbf{b}_k}{2}, \quad k = 0, 1, \ldots, 10, \tag{10}$$

where $\mathbf{a}_k$ and $\mathbf{b}_k$ are the k-th points on the top and bottom arcs respectively. The result is an 11-point line running down the centre of the worm's body — the spine.

2.5 Step 5: Measuring the Body Shape

From the spine, the two flanking arcs, and the contour, PySWIP computes 30 numbers per detected object per frame. The most important of these is the bending angle.

2.5.1 The Bending Angle θ

The bending angle θ measures how bent the worm is. It is computed from three spine points: the head end ($\mathbf{s}_0$), the midpoint ($\mathbf{s}_5$), and the tail end ($\mathbf{s}_{10}$):

$$\theta = \arccos\left(\frac{(\mathbf{s}_0 - \mathbf{s}_5) \cdot (\mathbf{s}_{10} - \mathbf{s}_5)}{\|\mathbf{s}_0 - \mathbf{s}_5\| \, \|\mathbf{s}_{10} - \mathbf{s}_5\|}\right). \tag{11}$$

The dot product divided by the product of lengths is a way of computing the cosine of the angle between two vectors. Taking the arccosine gives the angle itself. In plain terms: θ is the angle at the worm's midpoint, formed between the line to the head and the line to the tail.

- $\theta = 180°$: the worm is perfectly straight (head, midpoint, and tail are all in a line).
- $\theta < 180°$: the worm is bent. A worm at maximum thrash bends to roughly 90° or less.

As the worm thrashes, $\theta(t)$ oscillates up and down over time. The job of BATRR (Section 4) is to count those oscillations.

2.5.2 Other Shape Measurements

In addition to the bending angle, PySWIP records a range of other descriptors that are primarily used by PySWIPR to distinguish real worms from debris. The most important of these are summarised in Table 1.

Table 1: Selected PySWIP shape features and their biological or filtering significance.

Feature	What it measures and why it matters
`Angle` (θ)	Bending angle at the midpoint. Oscillates as the worm thrashes; used directly to compute thrash rate.
`Area`	Pixel count of the blob. Worms have a characteristic area; very small or very large blobs are noise or artefacts.
`Solidity`	Blob area divided by convex hull area. Always ≤ 1 for a real object; values > 1 are numerically impossible and flag bad data.
`Circularity`	How close the shape is to a circle. Worms are elongated so this is low; round debris has high circularity.
`Eccentricity`	How elongated the fitted ellipse is. High eccentricity = elongated = more likely to be a worm.
`Width_Taper_Ratio`	Whether the tips are narrower than the body. Worm tips are pointed (low ratio); debris tends to be blunt (high ratio).
`Tip_Angle_Head/Tail`	Sharpness of each tip. Pointed = small angle = worm-like.

2.6 Step 6: Keeping Track of Objects Across Frames

2.6.1 Why Tracking Is Necessary

PySWIP detects blobs independently in every frame. Without tracking, there is no way to know whether the blob in frame 47 is the "same" worm as the blob in frame 46. Tracking links detections across frames, so that each worm gets a consistent identity (an `ObjectID`) throughout its lifetime in the video. This is essential for computing a time series of θ for a single worm.

2.6.2 Matching Blobs to Existing Tracks by Overlap

Each existing track carries the binary pixel mask from the previous frame. To match new blobs to tracks, PySWIP computes how much each blob overlaps with each existing track's previous mask:

$$\mathrm{OvF}(B, M) = \frac{\text{number of pixels in both } B \text{ and } M}{\text{number of pixels in } B}. \tag{12}$$

This overlap fraction is computed for every possible (blob, track) pairing. Then the best-matching pair is assigned, that pair is removed, and the process repeats until no remaining pair has overlap ≥ 0.5. Blobs with no matching track spawn new candidate tracks; tracks with no matching blob increment a "missed frames" counter. A track that misses 15 consecutive frames is retired.

2.6.3 Preventing False New Detections

Two additional safeguards prevent spurious new `ObjectID`s:

Confirmation gate. A brand new detection does not immediately get an `ObjectID`. It must survive 3 consecutive frames before being confirmed. This eliminates single-frame noise blobs that would otherwise each inflate the object count.

Area continuity. Even if a blob overlaps a track perfectly, if its pixel area has dropped to less than 35% of the track's expected area (estimated by an EMA), the match is rejected. This prevents a tiny noise fragment from accidentally "claiming" a valid track.

2.6.4 Keeping Head and Tail Consistent

There is no practical reason that PySWIP must keep track of the head and tail of the worm (the mid-point bending angle remains the same whether or not the head and tail is flipped). However, it is good to have an elementary version of this code in case a future experiment warrants the need to keep the head and tail consistent. The furthest-pair heuristic (Equations 8–9) finds two tips but does not know which is the head and which is the tail. PySWIP keeps the assignment consistent frame-to-frame by comparing both possibilities against the previous frame:

$$c_{\text{same}} = \|\mathbf{h}_t - \mathbf{h}_{t-1}\| + \|\mathbf{t}_t - \mathbf{t}_{t-1}\| \quad \text{(keep current assignment)}, \tag{13}$$

$$c_{\text{flip}} = \|\mathbf{h}_t - \mathbf{t}_{t-1}\| + \|\mathbf{t}_t - \mathbf{h}_{t-1}\| \quad \text{(swap head and tail)}. \tag{14}$$

Whichever assignment has the lower total distance from the previous frame is chosen. If the worm is coiled (tips very close together) or the tips have jumped an implausibly large distance, the previous assignment is simply locked in unchanged.

2.7 Output: The Raw Data Spreadsheet

For every confirmed worm in every frame, PySWIP writes one row to `worm_results.csv`. If a well has no detection in a frame, a sentinel row with all measurements set to `NaN` ("not a number") is written to preserve the temporal structure of the data. The full column list is given in Table 2.

Table 2: PySWIP output CSV column schema.

Group	Column	Description
Identity	`Rel_Frame`	Frame number within the current time window
	`TimeID`	Which 10-minute window this frame belongs to
	`Well`	Which well on the plate (e.g. R0C1 = row 0, column 1)
	`ObjectID`	Which worm/object (`NaN` if nothing detected)
Body geometry	`Angle`	Bending angle θ in degrees
	`Area`	Body pixel area
	`Net_Spine_Len`	Straight-line head-to-tail distance
	`Spine_Arc_Len`	Actual length of the curved spine
	`Width_Mean/Std/CV`	Body width statistics along the spine
	`Width_Taper_Ratio`	How much narrower the tips are vs. the body
	`Width_Max`	Maximum body width
	`Spine_Curvature_Total`	Total bending along the spine
	`Spine_Curvature_Std`	Consistency of bending along the spine
Contour/boundary	`Perimeter`	Outline length
	`Circularity`	How round the shape is (1 = circle)
	`Solidity`	Fill fraction relative to convex hull
	`Convex_Hull_Area`	Area of the smallest convex shape enclosing the blob
	`Eccentricity`	How elongated the fitted ellipse is
	`Ellipse_Major/Minor/Angle`	Fitted ellipse dimensions and orientation
	`Bounding_Rect_AR`	Length-to-width ratio of minimum bounding rectangle
	`Boundary_Curv_Std`	Irregularity of the outline
	`Tip_Angle_Head`	Sharpness of the head tip
	`Tip_Angle_Tail`	Sharpness of the tail tip
Position	`Centroid_X/Y`	Centre of mass of the blob
	`Head_X/Y`	Head tip location
	`Tail_X/Y`	Tail tip location

3 PySWIPR: Cleaning the Data

What PySWIPR does in one sentence. PySWIPR reads the raw PySWIP spreadsheet and removes rows and objects that are clearly wrong, too short-lived to be a real worm, duplicates, or from time windows with too much missing data.

PySWIP is deliberately permissive: it records everything that looks remotely worm-like, including debris particles, air bubbles, and segmentation artefacts, so that no real worm is missed. PySWIPR is the cleaning step that removes the non-worm detections before counting begins. It applies four filters in sequence, each targeting a different type of bad data.

3.1 Stage 1: Remove Physically Impossible Values

Solidity is defined as the ratio of a blob's area to the area of the smallest convex shape that contains it (the convex hull). Because the convex hull always contains the blob, this ratio can never exceed 1.0 by definition — it is mathematically impossible. A value above 1.0 therefore indicates a numerical error in the contour calculation (typically from a contour that clips the edge of the well). These rows are deleted entirely:

```
solidity_filtered <- raw_data %>%
  filter(Solidity <= 1)
```

Listing 2: Stage 1: one line removes all physically impossible rows.

This operates on individual rows (single-frame detections) rather than whole objects, so one bad frame does not cause a whole worm track to be discarded.

3.2 Stage 2: Remove Short-Lived Objects

A real worm is present for the entire time window (up to 1800 frames at 30 fps over 10 minutes). Debris particles, transient reflections, or segmentation ghosts typically appear for only a handful of frames before disappearing. For each unique (`TimeID`, `Well`, `ObjectID`) triplet, the total number of frames it appears in is counted:

$$n_{\text{frames}} = \text{number of rows belonging to that object.} \tag{15}$$

Objects whose lifetime falls below the first quartile (Q_1) of all object lifetimes across the whole dataset are discarded:

$$\text{keep object if } n_{\text{frames}} \geq Q_1. \tag{16}$$

Using the first quartile rather than a fixed number is intentional: the threshold adapts automatically to whatever frame rate and video length the experiment used, without requiring the user to change a parameter. In practice, the bottom 25% of objects by lifetime are almost always noise.

```
et_df <- solidity_filtered %>%
  group_by(TimeID, Well, ObjectID) %>%
  summarise(totalframes = n())          # count frames per object

b25 <- as.numeric(summary(et_df$totalframes)[2])  # first quartile

et_df_filtered <- et_df %>%
  filter(totalframes >= b25)            # keep only long-lived objects
```

Listing 3: Stage 2: count how long each object lives, then drop the shortest-lived 25%.

3.3 Stage 3: Resolve Overlapping Tracks

Even after the above filters, two objects can sometimes coexist in the same well during the same time window — for instance, when a coiling worm briefly appears as two separate blobs, or when a piece of debris is tracked alongside a real worm. Because the SWIP assay uses one worm per well, any pair of overlapping objects must be resolved: one is the worm and one is debris.

Detecting overlap. Two objects in the same well and time window are considered to overlap in time if their active frame ranges intersect, which can happen in three unique ways. First, two objects i, j will overlap if the start frame of an object i is between the start and end frame of another object j. Second, similarly, two objects will overlap if the end frame of an object is between the start and end frame of another object. Third, similarly, two objects will overlap if the start and end frame of an object is between the start and end frame of another object. Mathematically, all scenarios can be described as such:

$$(t_j^s \leq t_i^s \leq t_j^e) \text{ (first scenario) } \vee \ (t_j^s \leq t_i^e \leq t_j^e) \text{ (second scenario) } = \tag{17}$$

$$(t_j^s \leq t_i^s \wedge t_i^s \leq t_j^e) \ \vee \ (t_j^s \leq t_i^e \wedge t_i^e \leq t_j^e) \tag{18}$$

where i, j are objects and t_k^s, t_k^e indicate start and end times for an object k, respectively. Equation 17 also accounts for the third case, since the third case is a special case where the first and second scenario are both true. Therefore, it needs not be described separately. In code, the following equivalent logic is applied to a data frame consisting of all possible combinations of objects:

```
filter(ObjectID != i.ObjectID & ((i.Frame_end <= Frame_end & i.Frame_end >=
    Frame_start) | (i.Frame_start <= Frame_end & i.Frame_start >=
   Frame_start)))
```

Listing 4: Overlapping objects logic

Deciding which is the worm. Empirically, we know that the standard deviation of the bending angle of a worm, paralyzed or not, is largely variable and significantly different from the standard deviation of the bending angle of a debris[1]. Therefore, for overlapping objects, we can simply choose to keep the the object with the greater standard deviation value for the angles:

$$\sigma_\theta = \text{SD}(\theta_t \text{ over all frames of the object}). \tag{19}$$

In each overlapping pair, the object with the *lower* σ_θ is classified as debris and removed.

3.4 Stage 4: Remove Data-Sparse Time Windows

Even after removing bad objects, some (well, time window) combinations may have very few remaining data points — for example, because the worm was out of frame or undetectable for most of the window. Counting thrashes from only a handful of frames would give an unreliable rate. A (well, time window) group is therefore discarded entirely if it contains fewer than 900 rows:

$$\text{keep group if } n_{\text{rows}} > 900. \tag{20}$$

Since a full 10-minute window at 30 fps contains $10 \times 60 \times 30 = 1800$ frames, 900 rows means at least 50% of the window had valid data. Groups below this threshold are considered too incomplete to trust.

[1] For evidence of this claim, check section 9.1

Checkpoint files. After each stage, PySWIPR writes the current data to a new CSV file (`worm_results_1solidity.csv` through `worm_results_4soliditytEtOlTp.csv`). This allows the researcher to inspect how many rows were removed at each stage and verify that the filtering is working as expected.

4 BATRR: Counting Thrashes from the Angle Signal

What BATRR does in one sentence. BATRR reads the cleaned bending-angle time series for each worm, computes how fast the angle is changing, counts the peaks and troughs in that speed signal, and converts the count into thrashes per minute.

4.1 Why Not Just Count Zero-Crossings of the Angle?

A thrash is intuitively one complete side-to-side swing. One might expect to count how many times the bending angle $\theta(t)$ crosses a central "neutral" value. The problem is that the central "neutral" value is slightly different for all worms, and very different for paralyzed worms, in which the "neutral" value depends on the physical shape of the worm when it is paralyzed. As such, normalization parameters is difficult to determine. Moreover, even once the raw angle data is normalized around a central value, the number of times the worm crosses the central value will not reflect true biological signal because a very slow thrash — that would otherwise be discarded as a non-movement — will be counted as a thrash by the algorithm, if it strictly counted central value movements. Furthermore, this approach will miss actual thrashes made by the worm if the worm does not make a full body bend that crosses the threshold (rather, it almost made a full body bend) that a human researcher would otherwise have counted as a thrash. Instead of accounting for many edge cases, BATRR instead works on the **angular velocity** — how fast θ is changing — which has a clear peak every time the worm reverses direction, regardless of where in its range it is bending.

4.2 Computing Angular Velocity

The angular velocity is the frame-to-frame change in angle divided by the change in frame number:

$$\dot{\theta}_t = \frac{\theta_t - \theta_{t-1}}{f_t - f_{t-1}}, \tag{21}$$

where f_t is the frame number at time step t. Dividing by the frame difference rather than just subtracting accounts for missing frames (which are common after PySWIPR filtering).

These quantities are computed in R using the `lag()` function, which shifts a column by one row so that the current and previous values can be subtracted:

```
thrash_df_2 <- data %>%
  group_by(TimeID, Well) %>%
  mutate(
    # velocity: change in angle / change in frame number
    Ang_vel = (Angle - lag(Angle)) / (Rel_Frame - lag(Rel_Frame)),
  ) %>%
  na.omit()   # drop the first row of each group (no "previous" to compare)
```

Listing 5: Computing angular velocity and acceleration per (well, window).

4.3 Counting Peaks in the Velocity Signal

Each time the worm completes one half-swing (bending from one side and starting to return), the angular velocity $\dot{\theta}$ passes through a local maximum (if bending in the positive direction) or a

local minimum (if bending in the negative direction)[2]. Counting these peaks and troughs gives the number of half-cycles, and dividing by two converts that to full oscillation cycles (thrashes).

Analogy. Think of a pendulum. The pendulum's speed is greatest at the bottom and zero at the extremes. Each time the speed peaks (at the bottom), one half-swing has been completed. Two peak-speed moments = one full back-and-forth = one thrash.

Peak detection uses the `pracma::findpeaks` function with two important parameters:

- **`minpeakheight` = 20 degrees/frame.** A velocity peak must be at least 20 deg/frame to count. This excludes the tiny wobbles of a paralysed worm, which never reach this speed, while counting the genuine directional reversals of a thrashing worm. This value also represents the minimum angular velocity the worm needs to exhibit for the researcher to count a valid thrash.
- **`minpeakdistance` = 8 frames.** Two peaks must be at least 8 frames apart. This prevents a single noisy spike from being counted as multiple thrashes. More importantly, this value represents the fastest thrash that a worm can exhibit when recorded with a 30 fps footage. Through experimentation, it was determined that the fastest worm thrash was completed at 0.27 seconds, which corresponds to 8 frames in a 30 fps footage.

Peak detection is run twice — once on $\dot{\theta}$ (catching positive peaks) and once on $-\dot{\theta}$ (catching negative peaks, i.e., troughs). The total thrash count is:

$$N_{\text{thrash}} = \left\lfloor \frac{N_{\text{peaks}}(\dot{\theta}) + N_{\text{peaks}}(-\dot{\theta})}{2} \right\rfloor, \tag{22}$$

where the floor operation ($\lfloor \cdot \rfloor$) rounds down to the nearest whole number, and the division by two converts half-cycles to full cycles.

```r
thrash_df_2 <- data %>%
  group_by(TimeID, Well) %>%
  # ... (velocity computation from above) ...
  na.omit() %>%
  summarise(
    Thrashes = (
      # count peaks in positive velocity
      length(findpeaks(Ang_vel,  minpeakheight=20, minpeakdistance=8)) +
      # count peaks in negative velocity (= troughs in original signal)
      length(findpeaks(-Ang_vel, minpeakheight=20, minpeakdistance=8))
    ) %/% 2    # %/% is integer division (equivalent to floor(x/2))
  )
```

Listing 6: Counting thrashes per (well, window) using bidirectional peak detection on angular velocity.

4.4 Converting Count to Rate

Different (well, window) groups may have different numbers of valid data rows after filtering. A group with 1800 rows and 30 thrashes has a very different thrash rate from a group with 900 rows and 30 thrashes. To make groups comparable, the thrash count is normalised to thrashes per minute:

$$\text{Thrash Rate (TPM)} = N_{\text{thrash}} \times \frac{1800}{n_{\text{frames}}}, \tag{23}$$

[2]These directions are arbitrary.

where n_{frames} is the number of valid rows in the group and 1800 is the number of frames in exactly one minute at 30 fps ($30 \times 60 = 1800$). Groups where `findpeaks` returned no result (because the worm was fully paralysed) are set to a thrash rate of zero.

```
thrash_df_final <- as_tibble(k) %>%
  mutate(`Thrash rate` = Thrashes / Frames * 1800)
thrash_df_final[is.na(thrash_df_final)] <- 0   # paralysed wells -> 0 TPM
fwrite(thrash_df_final, file = "thrash.csv")
```

Listing 7: Normalising thrash count to thrashes per minute.

The output `thrash.csv` contains one row per (`TimeID`, `Well`) with three columns: `Frames` (how many valid data points were available), `Thrashes` (raw count), and `Thrash rate` (thrashes per minute).

5 Implementation Details

5.1 Software Requirements

Table 3 lists the software libraries each component depends on and what each library is used for.

Table 3: Key software dependencies.

Component	Package	Role
PySWIP (Python)	`opencv-python`	Reading video frames, finding contours, morphological operations
	`numpy`	Array arithmetic and linear algebra
	`scipy`	Gaussian smoothing of contour points
	`pandas`	Writing the output CSV
PySWIPR (R)	`tidyverse`	Data manipulation pipeline
	`data.table`	Fast multi-key join operations
	`ggpubr`	Diagnostic plots
BATRR (R)	`tidyverse`	Data manipulation pipeline
	`data.table`	Fast join of frame count and thrash tables
	`pracma`	`findpeaks` peak detection function
	`ggpubr`	Result visualisation

5.2 Data Files Produced at Each Stage

Table 4 shows every file the pipeline produces and what it contains.

Table 4: Files produced by each stage of the pipeline.

File	Produced by	Contents
`worm_results.csv`	PySWIP	Raw measurements: one row per (frame, well, detected object); 30 columns of shape and position data
`worm_results_1solidity.csv`	PySWIPR Stage 1	After removing rows with impossible solidity values
`worm_results_2solidityEt.csv`	PySWIPR Stage 2	After removing objects that appeared in fewer frames than Q_1
`worm_results_3solidityEtOl.csv`	PySWIPR Stage 3	After resolving overlapping objects (keeping the one with higher angle variability)
`worm_results_4solidityEtOlTp.csv`	PySWIPR Stage 4	After removing time windows with fewer than 900 valid frames; this is the final input to BATRR
`thrash.csv`	BATRR	Final result: one row per (TimeID, Well) with `Frames`, `Thrashes`, and `Thrash rate` (TPM)

6 Pipeline Summary and TikZ Diagram

Step 1. **Fix the lighting** (PySWIP). Divide each well ROI by a blurred version of itself to flatten the illumination dome, then apply CLAHE to boost local contrast.

Step 2. **Build a background model** (PySWIP). Maintain a slowly updating average of what an empty well looks like. Freeze pixels under detected worms so they are never absorbed into the background. Initialise from a blurred reference frame so paralysed worms are visible from frame 1.

Step 3. **Detect the worm** (PySWIP). Subtract the background from the current frame. Apply a two-threshold flood-fill to grow strict seed regions into complete worm-shaped blobs.

Step 4. **Trace the body** (PySWIP). Extract the blob's contour, resample it to 120 even points, find the two tips, split into two arcs, and average them to get an 11-point midline spine.

Step 5. **Measure the shape** (PySWIP). Compute 30 geometric numbers including the bending angle θ, body width profile, and contour shape statistics. Write one row per detected worm per frame to `worm_results.csv`.

Step 6. **Clean the data** (PySWIPR). Remove physically impossible values, short-lived noise objects, duplicate overlapping tracks, and data-sparse time windows in four sequential filtering stages.

Step 7. **Count thrashes** (BATRR). Compute the angular velocity of the bending angle. Count peaks and troughs using a minimum height threshold of 20 deg/frame and minimum separation of 8 frames. Normalise to thrashes per minute. Write `thrash.csv`.

The next figure is a diagram for a visual representation of the pipeline.

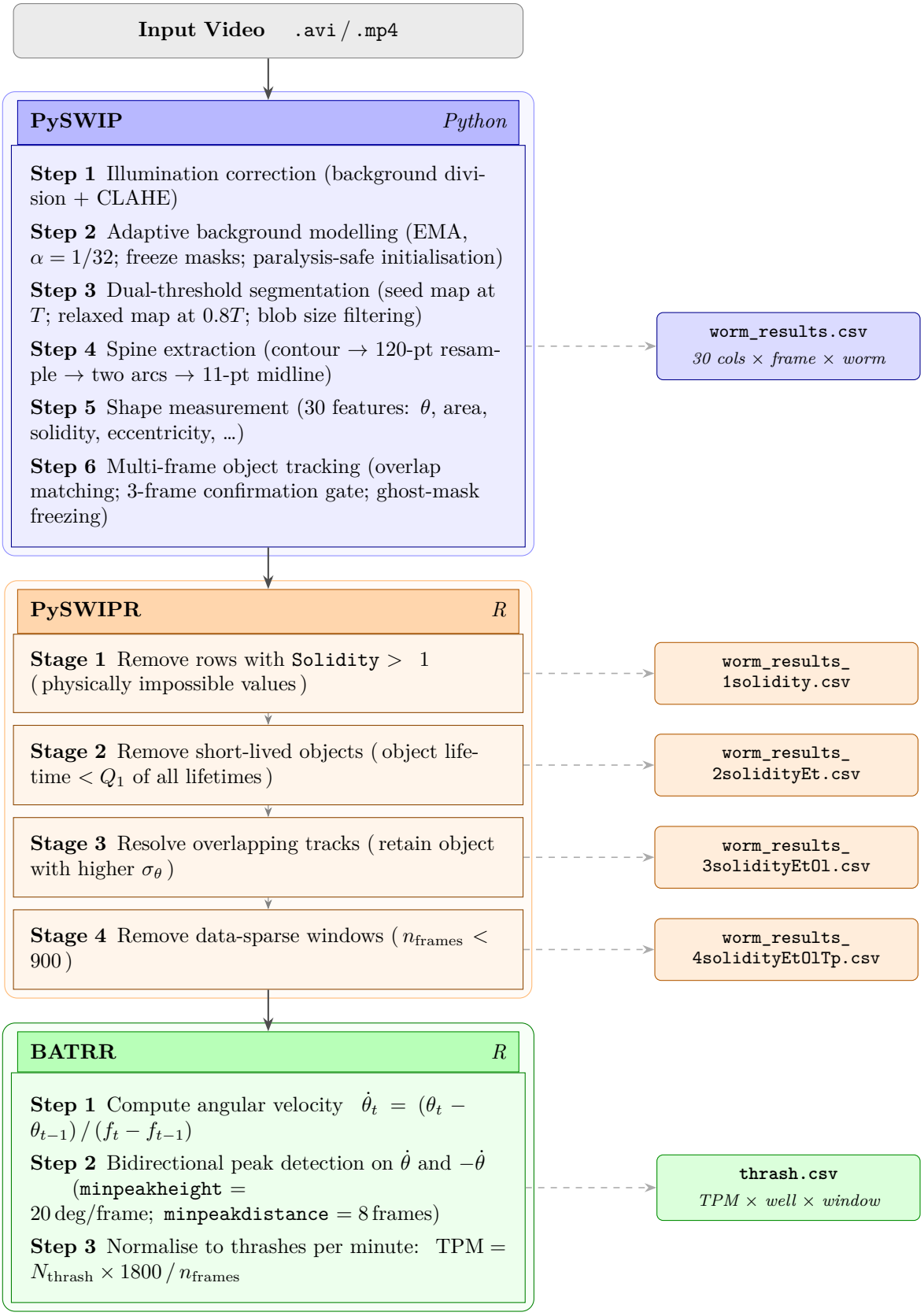


Figure 1: Full pipeline overview of the *C. elegans* SWIP Tracking Suite. Solid arrows indicate the main data flow between components; dashed arrows indicate checkpoint files written after each processing stage.

7 Pipeline Validation: Comparison with Manual Counting

To assess the performance of the *C. elegans* SWIP Tracking Suite, a simple experiment was conducted. 33 random (well, time interval) groups (time intervals of 1 or 9 were used as performance of this tracking suite is most important at the extreme ends of worm behavior) were chosen to assess the difference between the thrash rate counted by the algorithms in the tracking suite and the thrash rate counted manually through watching the worm footage. Statistical agreement between the two methods was evaluated through a multi-step approach. First, the Shapiro-Wilk test was used to assess the normal distribution of the differences between measurements. A standard linear regression analysis was then performed to determine the overall trend of the relationship. To further scrutinize the interchangeability of the methods, a Bland-Altman analysis was conducted to quantify systematic bias and limits of agreement. Finally, Passing-Bablok regression was employed as a robust, non-parametric alternative to assess for proportional and constant bias, providing a comprehensive view of method comparability.

7.1 Shapiro-Wilk Test for Normality

The difference between the manually-counted thrash rate and algorithm thrash rate were computed using R. This vector, containing all the differences, was used as input to the `shapiro.test()` function provided in the `stats` package. As the results yielded $p = 0.1752$, as per the convention, we failed to reject the null hypothesis that the distribution is normal.

7.2 Linear Regression Analysis

The figure below highlights the differences between the algorithm and manual count by plotting the thrash rates calculated manually and algorithmically on the x- and y-axis, respectively. This was done to assess the overall trend between the algorithm and manual calculations. Although using a linear correlation analysis has its limitations in assessing agreement between different quantitative methods[3], it is still provided to offer a traditional perspective.

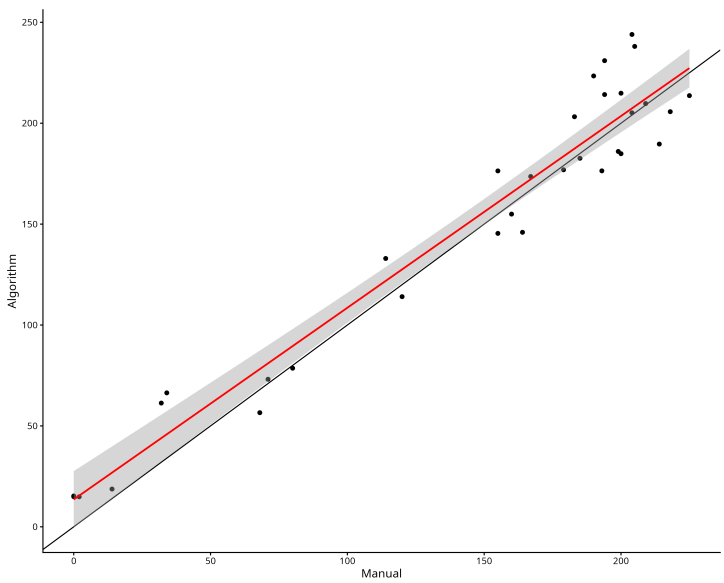


Figure 2: Red line is the regression line. Gray areas are standard error. Solid line represents $y = x$. Slope 95% CI: [0.8628107, 1.03729]; intercept 95% CI: [-0.4960364, 27.59098].

[3]The biggest criticism of doing so is that a linear model assumes that the x-axis, i.e. the independent variable, is perfectly measured. For many quantitative methods in laboratory analysis, this assumption is broken.

With an intercept of approximately 13.5 and slope of approximately 0.95, the linear model regression analysis demonstrates that, although algorithm tends to count higher thrash rates compared to the manual, with a slope of 0.95, it can work as a reasonable approximation to the manual count[4].

7.3 Bland-Altman Analysis

Bland-Altman Analysis is the gold standard for method comparison. It answers a simple question: Do these two scales/tools give the same answer? By calculating the average difference (bias) and the range of those differences (limits of agreement), we can see if the error between the two tools is small enough to ignore in a real-world setting. The following figure summarizes the results of the Bland-Altman Analysis.

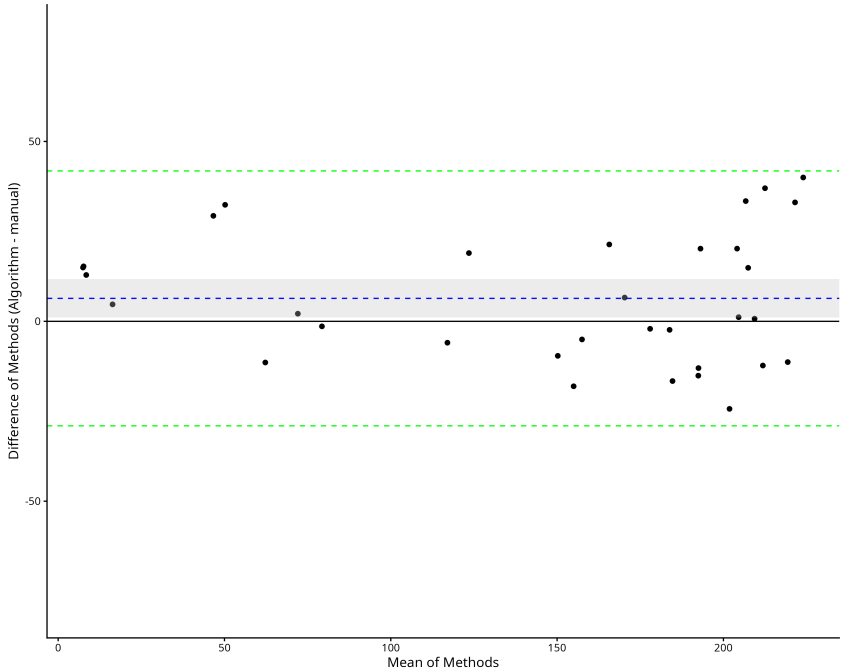


Figure 3: The dashed green lines represent the Limit of Agreement ([-29.04636, 41.81641]) of the bias (dashed black line, $y = 6.385023$). The solid black line represents 0 bias. The gray shaded area is the 95% CI of the bias ([1.054617, 11.71543]).

As the 95% CI of the bias does not cross 0, we can interpret this as our algorithm exhibiting a positive bias, where it systematically counts more thrashes than the manual count.
Critically, a Bland-Altman analysis quantifies agreement but does not inherently determine its analytical acceptability; such thresholds must be established a priori. Considering the documented inconsistencies in manual C. elegans thrash counting, an acceptable range of agreement was defined as ±40 thrashes (a total width of 80). This benchmark was derived from a separate study, where researchers observed discrepancies ranging from 80-100 thrashes between two human observers at high frequencies (machine count over 100; human count 90-250) [1]. Our observed limits of agreement fall within this established margin of human inter-observer variability.

[4] A shared slope suggests the underlying mechanism being measured is identical across groups. The y-intercept, by contrast, often reflects a baseline offset or starting condition that doesn't alter the trend itself. Strictly speaking, as the 95% CI of the intercept line crosses 0, it can be interpreted that there are no significant differences in the intercept.

7.4 Passing-Bablok Regression Analysis

Passing-Bablok regression analysis is a powerful non-parametric technique that does not assume that the variables follow a normal distribution. This regression analysis is popular in quantitative comparison of methods (especially in laboratory medicine), where two different analytical techniques may not exhibit the assumptions of a normal distribution. This technique addresses the downfalls of the linear regression analysis conducted in section 7.2. Much like a standard linear regression analysis, the 95% CI of the Passing-Bablok regression line is analyzed for interpretation. If the intercept's 95% CI includes 0, then it can be concluded that there is no significant constant difference between the two methods of measurement; similarly, if the slope's 95% CI includes 1, then it can be concluded that there is no significant proportional difference between the two methods of measurement [2]. The table below summarizes the result of the Passing-Bablok regression analysis.

Table 5: Passing-Bablok Regression Analysis Results

	Estimate (EST)	**SE**	**Lower 95% CI**	**Upper 95% CI**
Intercept	-1.5122216	10.95622686	-23.8575936	20.833150
Slope	0.9917113	0.07655039	0.8355857	1.147837

Note: Jackknife method used for confidence interval calculation.

From inspection of the 95% CI for both methods, it was concluded that there is no significant constant or proportional bias between the manual and SWIP Automation Suite thrash counting methods.

7.5 Algorithm Performance Assessment Summary

Collectively, the results of the four statistical analyses conducted suggest that the algorithm is a reasonable approximation of manual thrash counting. The Shapiro-Wilk test confirmed that the differences between the two methods are normally distributed ($p = 0.1752$), satisfying the assumptions of the Bland-Altman analysis. The linear regression analysis yielded a slope of approximately 0.95, with a 95% CI of [0.8628, 1.0373], suggesting a near-proportional relationship between the two methods. The Bland-Altman analysis revealed a positive bias of 6.39 thrashes (95% CI: [1.05, 11.72]), with limits of agreement of [-29.05, 41.82] — within the ± 40 thrash tolerance derived from documented human inter-observer variability. Finally, the Passing-Bablok regression analysis confirmed the absence of significant constant or proportional bias between the two methods. Taken together, these results suggest that the *C. elegans* SWIP Tracking Suite is a reproducible and reliable alternative to manual thrash quantification.

8 References

[1] S. D. Buckingham and D. B. Sattelle, "Fast, automated measurement of nematode swimming (thrashing) without morphometry," *BMC Neurosci*, vol. 10, p. 84, Jul. 2009, doi: 10.1186/1471-2202-10-84.

[2] L. Bilic-Zulle, "Comparison of methods: Passing and Bablok regression," *Biochem Med*, pp. 49–52, 2011, doi: 10.11613/BM.2011.010.

9 Supplementary Material

9.1 Standard Deviation of Angles: Worms and Debris

To determine whether the standard deviation of the angle between worms (paralyzed or thrashing) and debris were significantly different from each other, angle time series data was gathered from 18 debris, 9 paralyzed worms, and 9 thrashing worms, and the standard deviation of angle (θ_{SD}) were compared.

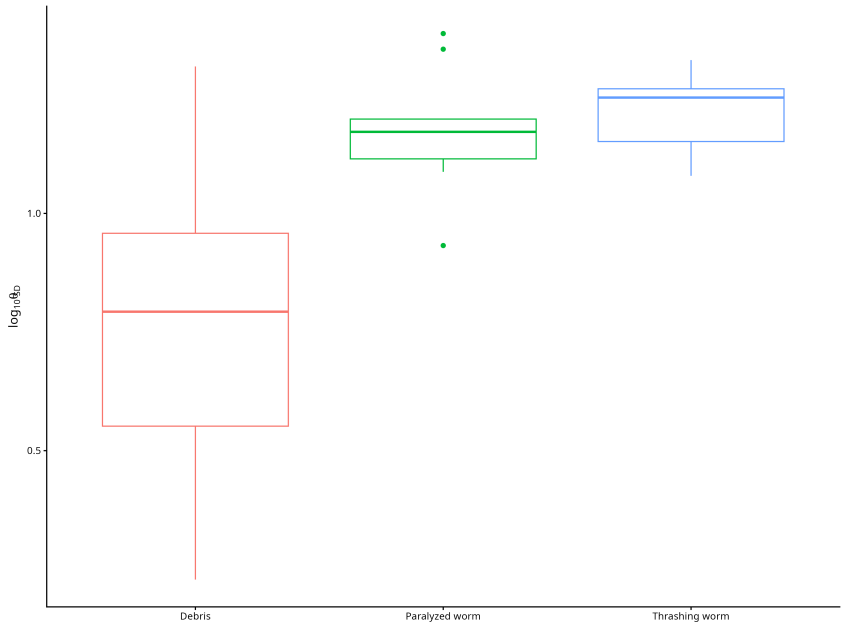
9.1.1 Shapiro-Wilk's Test for Normality

Raw θ_{SD} values were $\log_{10}$-transformed prior to analysis, as Shapiro-Wilk's test for normality was not satisfied for the debris group ($p = 0.004952$), but satisfied for the paralyzed and thrashing worm groups ($p = 0.349$ and $p = 0.6232$, respectively). Following $\log_{10}$- transformation, all groups satisfied the Shapiro-Wilk's test (p-values shown in table 9.1.5), satisfying the assumptions of an ANOVA test.

9.1.2 Outlier Identification and Retention

Three outliers were identified in the paralyzed worm group via boxplot inspection. As these outliers would inflate within-group variance, their retention represents a conservative approach — any bias would reduce statistical power to detect differences involving this group rather than inflate them. All observations were retained for analysis. Below is the boxplot with depiction of outliers.

Figure 4: Boxplot of Means of $\log_{10}(\theta_{\text{SD}})$



9.1.3 Levene's Test and Welch's ANOVA

As Levene's test indicated unequal variances across groups ($p = 0.004$), a Welch's one-way ANOVA was used to assess the relationship between object identity (debris, paralyzed worm, and thrashing worm) and the mean $\log_{10}$-transformed θ_{SD}, denoted $\overline{\log_{10}(\theta_{\text{SD}})}$, revealing a significant effect of identity ($p = 0.0002782$).

9.1.4 Games-Howell Post Hoc Analysis

A Games-Howell post hoc test was conducted given the unequal variances between groups, revealing that the $\overline{\log_{10}(\theta_{\mathrm{SD}})}$ of debris is significantly different from that of paralyzed and thrashing worms ($p_{\mathrm{adj}} = 0.000718$ and $p_{\mathrm{adj}} = 0.00013$, respectively). The $\overline{\log_{10}(\theta_{\mathrm{SD}})}$ of paralyzed worms was not significantly different from thrashing worms ($p_{\mathrm{adj}} = 0.772$). The next section summarizes the findings.

9.1.5 Statistical Analyses Overview

Table 6: Shapiro-Wilk's Normality Test

Identity	W	p-value
Debris	0.95998	0.6012
Paralyzed worm	0.94159	0.5988
Thrashing worm	0.92268	0.4149

Table 7: Levene's Test for Variance Homogeneity

Source	Df	F value	p-value
Identity	2	6.6813	0.003662

Table 8: Welch's One-Way ANOVA

Source	Num df	Denom df	F value	p-value
Identity	2	20.18	12.623	2.782×10^{-4}

Table 9: Games-Howell Post-hoc Comparisons

Contrast	Estimate	95% CI	p_{adj}
Debris vs. paralyzed worm	0.371	[0.155, 0.588]	0.000718
Debris vs. thrashing worm	0.408	[0.206, 0.609]	0.00013
Paralyzed worm vs. thrashing worm	0.036	[-0.102, 0.175]	0.772

Estimates are mean differences in $\log_{10}(\theta_{\mathrm{SD}})$.

9.1.6 Conclusion

In conclusion, after log-transformation of the standard deviation of angles to satisfy the assumptions of Welch's ANOVA test, it was concluded that the $\overline{\log_{10}(\theta_{\mathrm{SD}})}$ of a debris is meaningfully smaller than that of paralyzed and thrashing worms. Incidentally, it was also revealed that the $\overline{\log_{10}(\theta_{\mathrm{SD}})}$ of paralyzed and thrashing worms were not statistically different from each other. These conclusions demonstrate that the $\overline{\log_{10}(\theta_{\mathrm{SD}})}$ is a biologically-motivated and statistically-significant parameter to resolve overlapping object signals in PySWIP, whereby choosing the object with a greater $\overline{\log_{10}(\theta_{\mathrm{SD}})}$ value would result in most likely choosing a worm signal over a debris signal. This is equivalent to choosing the object with a greater $\overline{(\theta_{\mathrm{SD}})}$, as $\log_{10}$ is a strictly increasing transformation, the ordering of θ_{SD} values is preserved, and conclusions drawn on $\log_{10}(\theta_{\mathrm{SD}})$ apply directly to the untransformed θ_{SD}.